

References, Aliasing, and Mutation

Lecture 20

CS 1001

Kirk Duran

Today

- ▶ Review names, values, and immutable primitive data.
- ▶ Define mutability, identity, and aliasing precisely.
- ▶ Explain why assigning one list variable to another does not copy the list.
- ▶ Trace shared mutation, including mutation through function parameters.
- ▶ Contrast mutable lists with immutable strings and tuples.
- ▶ Create independent list copies when shared mutation is not desired.

Key idea

Assignment binds a name to an object. If two names refer to the same mutable object, a mutation through either name is visible through both.

Part 1: Values, Bindings, and Mutability

Names Are Bound to Objects

Example

```
x = 10  
y = x
```

- ▶ Python evaluates the right-hand side, then binds the resulting object to the name on the left.
- ▶ A name is not the object itself.
- ▶ Multiple names may refer to the same object.
- ▶ This distinction becomes important when the shared object is mutable.

Key idea

When tracing references, track both the **name** and the **object** to which it refers.

Mutable and Immutable Objects

Definition

An object is **mutable** if its observable state can be changed after creation. An object is **immutable** if its value cannot be changed in place after creation.

Immutable in this unit	Mutable in this unit
int, float, bool str, tuple	list

Key idea

Mutation changes an existing object. Reassignment changes which object a name refers to.

Immutable Values: Rebinding Instead of Mutation

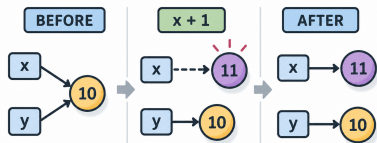
Code

```
x = 10  
y = x  
x = x + 1
```

Afterward

```
x → 11  
y → 10
```

The integer 10 was not changed into 11; a new value was produced and x was rebound.



Lists Introduce a New Possibility: In-Place Change

Code

```
scores = [90, 88, 76]  
scores[1] = 100
```

- ▶ The name `scores` still refers to the same list object.
- ▶ The state of that list object has changed.
- ▶ This is mutation, not rebinding.

Key idea

Once mutable data is shared, the distinction between changing a binding and changing an object becomes observable.

Part 2: Aliasing

Definition: Aliasing

Definition

Aliasing occurs when two or more names refer to the same object.

Example

```
scores = [90, 88, 76]  
backup = scores
```

Important

`backup = scores` does not duplicate the list. It creates another binding to the same list object.

Mental Model: Two Names, One List

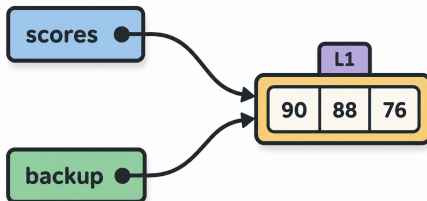
Bindings

scores ↘

backup ↗

[90, 88, 76]

- ▶ one list object;
- ▶ two references;
- ▶ mutation is shared.



Trace Shared Mutation

Statement	Bindings	Object state
a = [1, 2, 3]	a -> L1	L1 = [1, 2, 3]
b = a	a -> L1, b -> L1	L1 = [1, 2, 3]
b[0] = 9	a -> L1, b -> L1	L1 = [9, 2, 3]

Key idea

Do not say “b changed a.” The indexed assignment changed the shared list object.

Equality and Identity Answer Different Questions

Code

```
a = [1, 2]
b = a
c = [1, 2]
```

Expression	Result	Question
a == b	True	same contents?
a is b	True	same object?
a == c	True	same contents?
a is c	False	same object?

Prediction: Follow the Object, Not Just the Variable Name

Code

```
first = [4, 5, 6]  
second = first  
first[-1] = 0
```

In class

Draw the reference diagram after each statement. Then determine the values observed through `first` and `second`.

Part 3: Mutation as an Observable Side Effect

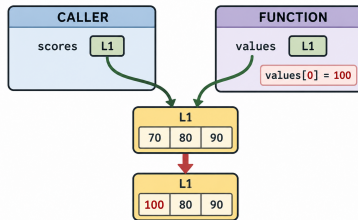
Function Parameters Can Alias Caller-Owned Lists

Code

```
def set_first(values: list, new_value:
int) -> None:
    values[0] = new_value

scores = [70, 80, 90]
set_first(scores, 100)
```

During the call, scores and parameter values refer to the same list.



Definition: Side Effect

Definition

A **side effect** is an observable change to program state that occurs in addition to producing a return value. Here, our main example is mutation of a list supplied by the caller.

- ▶ `set_first(...)` returns `None`, but changes the caller-visible list.
- ▶ Methods such as `append()`, `remove()`, and `reverse()` also intentionally mutate lists.
- ▶ Mutation is not inherently an error; unexpected or undocumented shared mutation is the problem.

Trace a Function Side Effect

Code

```
def replace_last(items: list, value: int) -> None:
    items[-1] = value

data = [2, 4, 6]
replace_last(data, 99)
```

In class

At the indexed assignment, identify every active name referring to the list object. What is the final value observed through data?

Part 4: Immutable Sequences — Strings and Tuples

Strings Prevent In-Place Element Mutation

Code

```
text = "cat"  
text[0] = "b"
```

Result

Python raises `TypeError`. A string does not support indexed assignment.

- ▶ Indexing can read a string position.
- ▶ Slicing and concatenation create new strings.
- ▶ The existing string value cannot be changed in place.

String “Change” Means Construct, Then Rebind

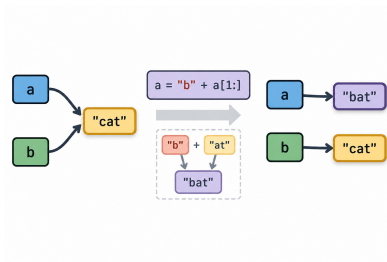
Code

```
a = "cat"  
b = a  
a = "b" + a[1:]
```

Afterward

```
a → "bat"  
b → "cat"
```

The original "cat" object was not mutated.



Definition: Tuple

Definition

A **tuple** is an ordered, indexed, immutable sequence of references.

Examples

```
point = (3, 5)    empty = ()    single = (42,)
```

<code>point[0]</code>	<code>3</code>
<code>point[:1]</code>	<code>(3,)</code>
<code>len(point)</code>	<code>2</code>
<code>5 in point</code>	<code>True</code>

To “Change” a Tuple, Build a New Tuple

Invalid mutation

```
data = (10, 20, 30)
data[1] = 99    →    TypeError
```

Construct and rebind

```
data = data[:1] + (99,) + data[2:]
data → (10, 99, 30)
```

Key idea

Slicing and concatenation produce a new tuple; assignment then rebinds the name.

Lists, Strings, and Tuples Share a Sequence Model

	List	String	Tuple
Ordered / indexed	yes	yes	yes
Sliceable	yes	yes	yes
len() / membership	yes	yes	yes
Element assignment	yes	no	no
Mutable	yes	no	no

Part 5: Creating Independent List Copies

When Aliasing Is Not What You Want

Problem

```
original = [10, 20, 30]  
working = original  
working[0] = 99
```

- ▶ `working` is an alias, not an independent working copy.
- ▶ Mutation is therefore visible through `original`.
- ▶ If independent mutation is required, deliberately create a new list object.

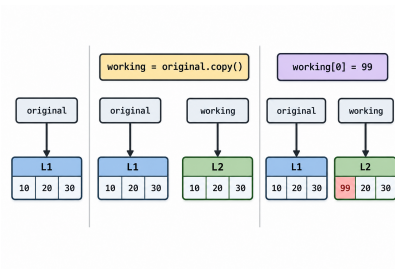
copy() Creates a New List Object

Code

```
original = [10, 20, 30]
working = original.copy()
working[0] = 99
```

Afterward

```
original → [10, 20, 30]
working → [99, 20, 30]
```



Other Ways to Create an Independent List Container

Full slice

```
working = original[:]
```

List construction

```
working = list(original)
```

- ▶ Both forms create a new list object.
- ▶ The new list initially contains equal element values.
- ▶ For the primitive-value lists used in this unit, mutation of positions in one list does not alter the other list.

Alias vs. Copy: Equality Can Be the Same While Identity Differs

Code

```
a = [1, 2, 3]
b = a
c = a.copy()
```

a == b		True	a is b		True
a == c		True	a is c		False

Key idea

Copying can preserve current contents while intentionally changing object identity.

Copy Before Mutation When Isolation Is Required

Example

```
def changed_first(values: list, new_value: int) -> list:
    result = values.copy()
    result[0] = new_value
    return result
```

- ▶ The parameter still refers to the caller's list.
- ▶ Mutation is performed only on the new list.
- ▶ The caller's original list remains unchanged.

Two Valid Function Designs

Mutate caller-owned list

```
def set_first(xs, value):  
    xs[0] = value
```

- ▶ intentional side effect;
- ▶ returns None.

Return a modified copy

```
def with_first(xs, value):  
    result = xs.copy()  
    result[0] = value  
    return result
```

- ▶ original stays unchanged;
- ▶ new list is returned.

Part 6: Integrated Tracing and Design Rules

Integrated Trace: Alias and Independent Copy

Code

```
a = [10, 20, 30]
b = a
c = a.copy()
b[1] = 99
c[0] = 5
```

In class

Determine the final values observed through a, b, and c. Draw the reference diagram after each statement.

Integrated Trace: Immutable Rebinding

Code

```
s1 = "code"  
s2 = s1  
s1 = "C" + s1[1:]  
  
t1 = (1, 2, 3)  
t2 = t1  
t1 = t1[:1] + (9,) + t1[2:]
```

In class

Determine the final values of all four names. Identify every newly constructed object.

Common Incorrect Mental Models

Incorrect	Replace it with
"Each variable stores its own copy."	A name refers to an object; names may alias one object.
"Changing b changed a."	Mutation changed the shared object.
"Reassignment mutates the old value."	Reassignment changes a binding.
"A string method edits the string."	String operations produce new string values.

A Practical Tracing Procedure

1. Identify each object that is created.
2. Record which names refer to each object.
3. For assignment, decide whether the result is a new object or an existing reference.
4. For a mutating operation, update the object itself.
5. For an immutable operation, record the newly produced value and any rebinding.
6. During a function call, check whether a parameter aliases caller-owned mutable data.

Choosing Between a List and a Tuple

Use a list when...

positions are expected to change
elements may be inserted or removed
mutation is part of the intended model

Use a tuple when...

the sequence should remain fixed
element replacement should be disallowed
immutability communicates the intended model

Operation Summary

Operation	Effect
<code>b = a</code> where <code>a</code> is a list	alias existing list
<code>a[i] = value</code>	mutate list object
<code>a.copy()</code> , <code>a[:]</code> , <code>list(a)</code>	create new list
string indexed assignment	invalid
tuple indexed assignment	invalid
string/tuple reconstruction + assignment	create new value, then rebind

Takeaways

- ▶ Names refer to objects; assignment does not automatically copy mutable data.
- ▶ Aliasing means multiple names refer to the same object.
- ▶ Mutation changes a mutable object, so every alias observes the new state.
- ▶ Function parameters can alias caller-owned lists, making mutation visible outside the call.
- ▶ Integers, floats, booleans, strings, and tuples are immutable; apparent changes require new values and rebinding.
- ▶ Tuples retain sequence operations while preventing element replacement.
- ▶ Use `copy()`, a full slice, or `list(...)` when an independent list is required.

Key idea

When tracing mutable data, ask not only “What value does this variable show?” but also “Which object does this name refer to?”